

TACG: Trajectory-Aware Commit Gating for Diffusion Language Model Decoding

Chengcheng Wang^{1*} Tingzhang Luo^{2*} Wenhao Li¹ Jianyuan Guo² Chang Xu¹

¹University of Sydney ²City University of Hong Kong

cwan0785@uni.sydney.edu.au tingzluo3-c@my.cityu.edu.hk

weli0055@uni.sydney.edu.au jianyguo@cityu.edu.hk c.xu@sydney.edu.au

Abstract

Diffusion language models (DLLMs) generate text by iteratively denoising masked positions, exposing a trajectory of predictive distributions rather than a single instantaneous belief. Most existing decoders ignore this trajectory and commit tokens from the current snapshot alone, conflating confidence with *commitment readiness*: a transient top-1 peak under incomplete context can be locked in, while candidates with consistent cross-step support are delayed. We propose Trajectory-Aware Commit Gating (TACG), a training-free gate-level decoder that anchors token identities to the base posterior and uses trajectory-aware signals only to decide whether the current proposal is ready to commit. TACG combines Temporal Implicit Logits Guidance (TILG), which keeps an exponential moving average of past logits as a self-reference and contrasts the current logits against this reference in natural-parameter space, with a History Gate (HG) that enforces short-term proposal persistence before commitment. Together with a capped extra-promotion budget, these components yield a stability-constrained commit rule without auxiliary networks or extra forward passes. We evaluate TACG on LLaDA, Dream, and LLaDA2-Mini across code (HumanEval, MBPP) and math (GSM8K, MATH500) benchmarks; it typically improves or preserves accuracy while reducing denoising steps and increasing tokens per forward (TPF). The code is publicly available at <https://github.com/Clarence-CV/TACG-DLLM>.

1 Introduction

Diffusion language models generate text by reversing a masking process [2, 30, 21, 23]. Starting from a sequence in which many positions are masked, the model repeatedly predicts the masked positions and progressively commits a subset of them to concrete tokens. Unlike autoregressive models, which extend a sequence one token at a time, a single DLLM forward pass produces predictive distributions for many positions simultaneously. Decoding is therefore not only a token prediction problem, but also a decision about which predictions are reliable enough to enter the partially generated sequence.

As denoising proceeds, the logits at a position evolve with the surrounding context: the first-ranked (top-1) candidate, its confidence, and the distributional shape may all change. Each masked position thus carries a temporal belief trajectory, not merely an instantaneous prediction.

Most current DLLM decoders make limited use of this trajectory. Confidence-based remasking commits the highest-confidence positions at each step. Schedule-based decoders, inherited in part from masked generative modeling [4], reveal a prescribed number of positions regardless of the current model state. Stability-aware decoders such as KLASS [15] commit positions whose predictive distributions appear locally stable. These rules are largely snapshot-based: the decision at step t is

*Equal contribution.

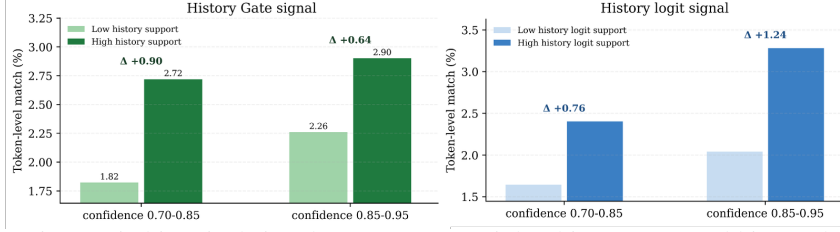


Figure 1: Diagnostic historical signals on GSM8K. Higher history gate and history-logit support improve token-level match, suggesting their roles in both reliable commitment and early token promotion.

determined by the current distribution p_t , possibly augmented with a local stability statistic. The model produces a trajectory, but the decoder consumes only its latest state.

This snapshot view is insufficient for training-free DLLM decoding, where the goal is to improve the quality–efficiency trade-off without changing model parameters. A decoder should avoid unreliable early commitments, since an incorrectly unmasked token becomes part of the conditioning context for subsequent denoising steps. At the same time, it should reveal ready tokens as early as possible, since delaying them increases the number of denoising steps and reduces parallelism. Simply adjusting the confidence threshold cannot resolve this tension: a stricter threshold improves caution but delays ready tokens, while a looser threshold accelerates decoding but admits unstable candidates.

This raises a natural question: can a training-free decoder estimate commitment readiness from signals already available in the denoising process, rather than by retraining the model or adding lookahead? We find that the historical trajectory provides such signals. As shown in Figure 1, our GSM8K diagnostic analysis reveals two phenomena, with details in Section 2. History Gate support improves token-level match under comparable confidence, indicating a reliability cue beyond snapshot confidence. History-logit support is especially useful in lower-confidence regions, suggesting that historical logits can identify ready-to-commit tokens before a rigid confidence threshold would reveal them. Together, these signals support both reliable commitment and early token promotion.

Building on this observation, we separate two decisions that snapshot confidence often merges: *which token* should be written and *when* that token should leave the masked state. Instantaneous confidence measures the height of the current posterior peak, but it does not indicate whether that peak is transient, stalled, or supported by the denoising trajectory. We therefore keep token identity anchored to the base posterior and use the historical trajectory only to estimate commit readiness. Concretely, the base model proposes $\hat{x}_i = \arg \max_v p_t(i, v)$ with confidence $c_i = p_t(i, \hat{x}_i)$, and TACG decides whether this proposal should be committed now.

We introduce Trajectory-Aware Commit Gating (TACG) as a gate-level decoding framework. TACG combines two complementary trajectory-aware components. Its temporal-support branch, Temporal Implicit Logits Guidance (TILG), keeps an exponential moving average $\bar{z}_{t-1}$ of past logits as an implicit self-reference and uses the contrast between the current belief and this reference to compute temporal support for the base proposal. Its stability branch, the History Gate (HG), requires short-term proposal persistence before a position is committed, with a confidence escape for already-saturated positions. For compact notation, define the TILG auxiliary readout

$$q_t(v) \propto p_t(v) \left(\frac{p_t(v)}{p_t^{\text{ref}}(v)} \right)^w = \frac{p_t(v)^{1+w}}{p_t^{\text{ref}}(v)^w}. \quad (1)$$

Operationally, the TILG branch queries this readout only at the base proposal $\hat{x}_i$: if the proposal’s support has increased relative to the historical reference, its readiness score rises; if the current posterior peak is transient or weakening, the temporal support remains small. The contrast is over denoising time within a single model, and the reference is the model’s own slow belief obtained from logits already produced during decoding. TILG therefore requires no auxiliary network and no additional model forward pass.

To obtain a practical decoder, TACG couples this temporal signal to a stability-constrained commit rule. The HG suppresses transient candidates by requiring a short persistence window before commitment, while a confidence floor excludes clearly unready candidates. Beyond the stable base accept set, TACG promotes a bounded number of additional positions with the largest readiness scores $s_i = c_i + \lambda \sigma_i$, where c_i is the proposal confidence and σ_i is the temporal support of the

proposed token. The cap K_{extra} acts as an explicit acceleration budget: temporal evidence may move the reveal boundary, but only within a controlled number of additional commits per step.

Our contributions are as follows. First, we identify commit timing as a distinct decoder decision in DLLMs and show why instantaneous confidence is an incomplete proxy for commitment readiness. Second, we introduce TACG, a gate-level framework that separates token identity from commit timing and combines TILG temporal support with an HG persistence constraint. Third, we couple these components to a stability-constrained acceleration rule with capped, consistency-conditioned extra promotion. Fourth, we evaluate TACG on LLaDA [23], Dream [41], and LLaDA2-Mini [3] across code [5, 1] and math [7, 11] benchmarks, demonstrating consistent accuracy improvements and efficiency gains under confidence-based gates.

2 Preliminary

Masked Diffusion Language Models (MDLMs). MDLMs cast text generation as a discrete denoising problem over sequences. Let $x_0 = (x_0^1, \dots, x_0^L) \in \mathcal{V}^L$ be a clean token sequence of length L , with vocabulary $\mathcal{V}$. During the forward noising process, tokens in x_0 are gradually corrupted into a special mask symbol [MASK]. Given a partially corrupted sequence x_t at noise level $t \in [0, 1]$, the model learns to predict the original clean tokens at the masked positions. A commonly used MDLM training objective can be written as

$$\mathcal{L}_{\text{MDLM}}(\theta) = -\mathbb{E}_{x_0, t, x_t} \left[\frac{1}{t} \sum_{i=1}^L \mathbf{1}[x_t^i = \text{[MASK]}] \log p_\theta(x_0^i | x_t) \right]. \quad (2)$$

At inference time, generation typically begins from a fully masked sequence. The model then repeatedly predicts tokens for masked positions in parallel, optionally masking uncertain predictions again to allow further refinement.

Diagnostic Historical Signals. We analyze two historical signals on the GSM8K diagnostic subset to study whether commitment readiness can be better estimated beyond snapshot confidence. Gate-level persistence measures whether a proposal remains stable across steps, while logit-level reinforcement measures whether the proposal gains support from the historical logit trajectory.

• **History Gate Support.** We first examine a discrete form of historical evidence: whether the same token proposal persists across adjacent denoising steps. For each masked position i , we define the *History Gate support* as the consecutive number of steps for which the current proposal remains unchanged:

$$\ell_{t,i} = \begin{cases} \ell_{t-1,i} + 1, & \hat{x}_{t,i} = \hat{x}_{t-1,i}, \\ 1, & \hat{x}_{t,i} \neq \hat{x}_{t-1,i}. \end{cases} \quad (3)$$

A larger $\ell_{t,i}$ indicates that the proposal has persisted longer along the denoising trajectory. Figure 1(left) shows that candidates with higher History Gate support achieve higher token-level match within matched confidence regions. This suggests that proposal persistence provides a reliability cue beyond snapshot confidence. In other words, even when two candidates have similar confidence, the one that remains stable across recent denoising steps is more reliable for commitment. This motivates using History Gate support as a gate-level signal to improve commitment quality.

• **History-Logit Support.** While gate-level persistence measures whether a proposal is stable, improving decoding efficiency also requires identifying candidates that can be committed before reaching a rigid confidence threshold. Simply lowering the confidence threshold can reveal more tokens earlier, but it treats all lower-confidence candidates equally and may introduce unstable commitments. We therefore examine a continuous logit-level signal from the denoising trajectory.

Let $\bar{z}_{t-1}(i, \cdot)$ be an EMA reference of previous logits at position i . Given current logits $z_t(i, \cdot)$, we form a history-guided readout

$$q_t(i, \cdot) = \text{softmax}(z_t(i, \cdot) + \lambda [z_t(i, \cdot) - \bar{z}_{t-1}(i, \cdot)]), \quad (4)$$

where λ controls the strength of the historical signal. For the current proposal $\hat{x}_{t,i}$, we measure its history-logit support by the gain assigned to this proposal:

$$g_{t,i} = q_t(i, \hat{x}_{t,i}) - p_t^{\text{ref}}(i, \hat{x}_{t,i}), \quad p_t^{\text{ref}}(i, \cdot) = \text{softmax}(\bar{z}_{t-1}(i, \cdot)). \quad (5)$$

This signal is candidate-conditioned: it scores whether the current base proposal is reinforced by the historical logit trajectory, rather than replacing it with a new token. Figure 1(right) shows that

candidates with higher history-logit support achieve higher token-level match, especially in lower-confidence regions. This indicates that some candidates below a rigid confidence threshold are already supported by the denoising trajectory. Thus, history-logit support provides a logit-level signal for early token promotion: it can expand the commit boundary selectively, rather than uniformly relaxing the confidence threshold.

3 Related Work

Diffusion Language Models. Diffusion models [13, 32] have become dominant in visual generation [28, 25, 29, 46], and recent work has explored their application to text generation. Among existing paradigms, masked diffusion language models (MDLMs) [31, 2, 30, 50, 21] have emerged as a promising alternative to AR-LLMs by modeling language in discrete space through masked token prediction. Building on this formulation, LLaDA [23] and Dream [41] scale MDLMs to the billion-parameter regime with large-scale pretraining, demonstrating their practical potential. LLaDA-2.0 [3] and LLaDA-MoE [54] further show that MDLMs can be effectively scaled with mixture-of-experts architectures. Beyond these developments, dLLMs are also attracting increasing attention in reasoning [53, 24, 37, 27, 33, 22, 48], multimodal tasks [43, 44, 40, 42, 20, 34, 45, 6], code generation [38, 9, 8], long-context modeling [19, 10, 51], and agent [49, 47].

Training-Free Decoding for dLLMs. A growing line of work accelerates dLLM inference without updating model parameters. Fast-dLLM [35] combines block-wise approximate KV caching with confidence-aware token selection, while APD [14] adaptively controls parallel sampling. Other methods improve commitment or unmasking decisions using different signals: Prophet [18] uses top-2 margin for early commitment, LocalLeap [16] exploits local determinism around high-confidence anchors, LookUM [17] verifies candidate unmasking paths, and KCLASS [15] measures token-level stability with KL divergence and confidence. Recent works further explore lookahead branches [39], streaming-style pruning and early exit [36], and attention-based decoding orders [52]. Our method is orthogonal to these approaches: instead of designing a new schedule or lookahead mechanism, we exploit the model’s historical logit trajectory as a lightweight readiness signal for deciding when the current base proposal should be committed.

Relation to classifier-free guidance (CFG). The auxiliary readout used by the TILG branch, $q_t(v) \propto p_t(v)^{1+w}/p_t^{\text{ref}}(v)^w$ (Eq. 1), has the same algebraic shape as classifier-free guidance [12], which combines a conditional and an unconditional model. Despite this formal similarity, the two methods use the contrast differently. CFG applies the contrasted distribution *directly for sampling*, so the guidance weight modifies which token is produced. In the TILG component of TACG, the contrast is between the current logits and the model’s own EMA over past logits, and the resulting q_t is used *only to score commit readiness*: token identity remains $\arg \max_v p_t(v)$, while q_t gates *when* that token leaves the masked state. The guidance signal therefore changes the unmask boundary rather than the selected token.

4 Method

4.1 DLLM decoding as base proposal and commit gate

Let $\mathcal{M}_t$ denote the set of positions that remain masked at denoising step t . For each $i \in \mathcal{M}_t$, the DLLM produces logits $z_t(i, \cdot)$ and a predictive distribution

$$p_t(i, \cdot) = \text{softmax}(z_t(i, \cdot)). \quad (6)$$

The decoder must make two coupled decisions. The stopping decision $a_{t,i} \in \{0, 1\}$ determines whether position i is committed at step t , and the labeling decision $y_{t,i} \in \mathcal{V}$ determines which token is written if the position is committed. The full decoding action at step t is therefore $\mathcal{A}_t = \{(a_{t,i}, y_{t,i}) : i \in \mathcal{M}_t\}$. Snapshot-based decoders typically derive both decisions from p_t : the label is $\arg \max_v p_t(i, v)$, and the stopping decision compares $\max_v p_t(i, v)$ against a threshold or reveal schedule. Once $a_{t,i} = 1$, the selected token enters the conditioning context for subsequent steps; in non-revision decoders, an early incorrect commitment is not later corrected.

The central premise of our method is the following: given a strong base posterior, deciding *when* to commit a token is more fragile than deciding *which* token to write. We therefore anchor token identity to the base posterior,

$$\hat{x}_i = \arg \max_v p_t(i, v), \quad c_i = p_t(i, \hat{x}_i), \quad (7)$$

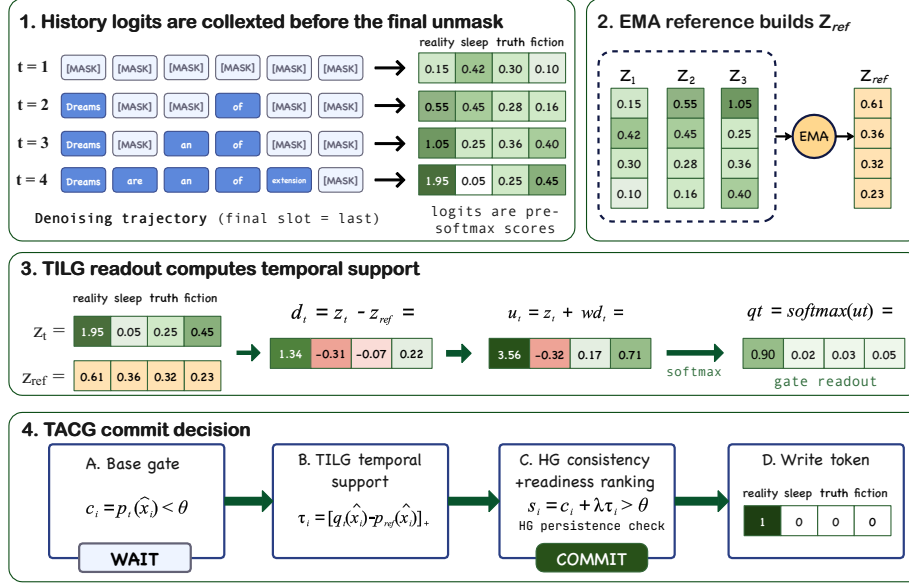


Figure 2: TACG mechanism with a TILG temporal-support branch. The base posterior first proposes the token identity $\hat{x}_i = \arg \max_v p_t(v)$. TILG then builds an EMA self-reference z_{ref} from the same denoising trajectory, contrasts the current logits with this reference, and uses the auxiliary readout $q_t = \text{softmax}(u_t)$ only to score commitment readiness. The written token remains the base proposal; TACG changes when a position is unmasked, not which token is written.

and use TACG only to decide whether this proposal should leave the masked state. A high-confidence top-1 proposal at step t need not remain first-ranked after additional context is revealed; some DLLM formulations admit revision [21, 30], but the decoders we build on are non-revising, and an early incorrect commitment is not later corrected. Estimating commitment readiness therefore requires information about the trajectory that produced the current belief, not only the current confidence value. In the default decoder, if the gate commits position i , the written token remains $\hat{x}_i$.

4.2 TILG: temporal self-reference in logit space

For categorical distributions, logits are natural-parameter coordinates: for any two tokens a and b , $\log(p_t(a)/p_t(b)) = z_t(a) - z_t(b)$. Differences in logit space therefore correspond directly to log-odds shifts. TILG maintains a slow reference $\bar{z}_t$ in logit space using an exponential moving average,

$$\bar{z}_t = \beta \bar{z}_{t-1} + (1 - \beta) z_t, \quad (8)$$

where β is a decay parameter. When a block or position is first observed, we initialize $\bar{z}_{t-1}$ with the current logits, making the temporal signal neutral at the first step. The contrast between the current logits and the reference,

$$d_t = z_t - \bar{z}_{t-1}, \quad (9)$$

captures the directional change of the predictive belief in natural-parameter space. We refer to d_t as the temporal belief contrast or belief innovation. It is not a strict time derivative, because $\bar{z}_{t-1}$ is an EMA reference rather than a single previous state.

4.3 TILG: temporal support for the base proposal

Given the current posterior p_t and the reference posterior $p_t^{ref} = \text{softmax}(\bar{z}_{t-1})$, TILG computes the auxiliary readout in Eq. 1 with temporal contrast weight $w \geq 0$. This readout is used only for scoring the base proposal. For any pair of tokens a and b , its log-odds satisfy

$$\log \frac{q_t(a)}{q_t(b)} = \log \frac{p_t(a)}{p_t(b)} + w \left[\log \frac{p_t(a)}{p_t(b)} - \log \frac{p_t^{ref}(a)}{p_t^{ref}(b)} \right]. \quad (10)$$

This identity makes the temporal-support interpretation explicit: the second term is the log-odds change from the historical reference to the current belief. The gate therefore gives higher support to a

base proposal whose relative evidence is increasing along the trajectory, and lower support to one whose evidence is weakening.

A small- w expansion gives a complementary view. Writing $q_w = \text{softmax}(z_t + wd_t)$ and differentiating at $w = 0$ yields

$$\left. \frac{\partial q_w(v)}{\partial w} \right|_{w=0} = p_t(v) (d_t(v) - \mathbb{E}_{u \sim p_t}[d_t(u)]), \quad (11)$$

Thus, increasing w raises the readout value of tokens whose belief innovation exceeds the current posterior average and lowers the readout value of tokens whose innovation is below that average. TILG is also invariant to the additive gauge of logits: shifting z_t and $\bar{z}_{t-1}$ by token-independent constants leaves q_t unchanged, because the induced shift in the auxiliary readout is token-independent and is absorbed by normalization.

4.4 TILG: candidate-conditioned temporal support

We use a bounded score to measure how strongly the recent trajectory supports the base proposal $\hat{x}_i$. The reported implementation uses a probability-gain form,

$$b_i = [q_t(i, \hat{x}_i) - p_t^{\text{ref}}(i, \hat{x}_i)]_+, \quad (12)$$

which is naturally bounded and numerically stable. A more direct log-ratio variant is

$$g_i = \text{clip}(\log(p_t(i, \hat{x}_i) + \epsilon) - \log(p_t^{\text{ref}}(i, \hat{x}_i) + \epsilon), 0, \gamma), \quad (13)$$

which follows directly from the ratio used to define q_t and avoids some saturation effects of probability-space gains at high confidence. We denote either choice generically by σ_i and treat g_i as a reference variant for ablation; the default in our experiments is b_i . Both scores are candidate-conditioned: they evaluate whether the base proposal has gained signed support, not which vocabulary item should replace it.

4.5 TACG commit rule: HG and capped extra promotion

The decoder builds on a base accept set $\mathcal{R}_t^{\text{base}} \subseteq \mathcal{M}_t$ produced by an underlying gate, such as a confidence threshold or KLASS [15]. To reduce transient commitments, TACG applies a proposal-consistency gate. This History Gate (HG) is the persistence component of TACG. Let h_i be the number of consecutive steps for which the base proposal at position i has remained unchanged. Define

$$H_i^{\text{base}} = \mathbf{1}[h_i \geq m_{\text{base}}] \vee \mathbf{1}[c_i \geq \tau_{\text{esc}}] \vee \mathbf{1}[\text{preserve_base}(i)], \quad (14)$$

where m_{base} is the required proposal-persistence length, τ_{esc} is a confidence escape for already-saturated positions, and $\text{preserve_base}(i)$ is an optional flag that preserves high-margin base decisions. The stable accept set is $\mathcal{B}_t = \{i \in \mathcal{R}_t^{\text{base}} : H_i^{\text{base}} = 1\}$.

Beyond $\mathcal{B}_t$, TACG considers additional promotions only among positions that pass a confidence floor and a lighter consistency condition,

$$H_i^{\text{extra}} = \mathbf{1}[h_i \geq m_{\text{extra}}] \vee \mathbf{1}[c_i \geq \tau_{\text{esc}}], \quad m_{\text{extra}} \leq m_{\text{base}}, \quad (15)$$

$$\mathcal{N}_t = \{i \in \mathcal{M}_t : i \notin \mathcal{B}_t, c_i \geq \tau_{\text{floor}}, H_i^{\text{extra}} = 1\}. \quad (16)$$

Candidates in $\mathcal{N}_t$ are ranked by the readiness score $s_i = c_i + \lambda \sigma_i$, and at most K_{extra} of them are committed:

$$\mathcal{E}_t = \arg \max_{\mathcal{E} \subseteq \mathcal{N}_t, |\mathcal{E}| \leq K_{\text{extra}}} \sum_{i \in \mathcal{E}} s_i, \quad (17)$$

so that $\mathcal{E}_t$ contains the top- K_{extra} positions in $\mathcal{N}_t$ according to s_i . The full gate at step t is

$$\mathcal{G}_t(i) = \mathbf{1}[i \in \mathcal{B}_t \cup \mathcal{E}_t]. \quad (18)$$

The token written by this gate is still the base proposal,

$$x_{\text{write},i} = \begin{cases} \hat{x}_i, & \mathcal{G}_t(i) = 1, \\ [\text{MASK}], & \text{otherwise.} \end{cases} \quad (19)$$

The cap K_{extra} has a clear interpretation as a per-step acceleration budget: TACG may promote temporally supported candidates beyond the base reveal boundary, but the number of additional

Table 1: Main comparison of TACG on LLaDA and Dream. Rows with TACG apply the full trajectory-aware commit gate to the corresponding base decoder; parentheses show changes relative to the baseline decoder. d3LLM[†] denotes reported results from prior work [26].

(a) LLaDA					(b) Dream				
Dataset	Method	Acc. $\uparrow$	Steps $\downarrow$	TPF $\uparrow$	Method	Acc. $\uparrow$	Steps $\downarrow$	TPF $\uparrow$	
HumanEval	Conf.	37.20	52.28	6.45	Conf.	47.56	51.19	5.00	
	Conf. + TACG	42.07 (+4.87)	45.71 (-6.57)	7.01 (+0.56)	Conf. + TACG	47.56 (+0.00)	49.45 (-1.74)	5.18 (+0.18)	
	d3LLM [†]	39.60	–	5.95	d3LLM [†]	57.10	–	3.20	
	KLASS	39.63	92.13	3.34	KLASS	60.37	75.09	3.41	
	KLASS + TACG	40.85 (+1.22)	88.30 (-3.83)	3.49 (+0.15)	KLASS + TACG	60.37 (+0.00)	72.05 (-3.04)	3.55 (+0.14)	
MBPP	Conf.	47.08	83.34	3.44	Conf.	57.20	73.73	3.47	
	Conf. + TACG	49.03 (+1.95)	74.76 (-8.58)	3.83 (+0.39)	Conf. + TACG	57.59 (+0.39)	70.93 (-2.80)	3.61 (+0.14)	
	d3LLM [†]	40.60	–	4.21	d3LLM [†]	55.60	–	2.96	
	KLASS	47.86	121.08	2.30	KLASS	61.87	111.25	2.30	
	KLASS + TACG	49.81 (+1.95)	119.70 (-1.38)	2.33 (+0.03)	KLASS + TACG	62.65 (+0.78)	82.17 (-29.08)	3.12 (+0.82)	
MATH500	Conf.	30.60	96.48	3.37	Conf.	39.00	94.60	2.71	
	Conf. + TACG	32.20 (+1.60)	86.59 (-9.89)	3.78 (+0.41)	Conf. + TACG	40.00 (+1.00)	80.33 (-14.27)	3.19 (+0.48)	
	d3LLM [†]	30.40	–	5.74	d3LLM [†]	38.20	–	3.92	
	KLASS	30.60	127.70	2.52	KLASS	40.60	148.85	1.72	
	KLASS + TACG	33.80 (+3.20)	133.33 (+5.63)	2.43 (-0.09)	KLASS + TACG	40.60 (+0.00)	145.79 (-3.06)	1.76 (+0.04)	
GSM8K	Conf.	75.40	74.26	3.68	Conf.	73.16	74.63	3.43	
	Conf. + TACG	77.33 (+1.93)	72.29 (-1.97)	3.78 (+0.10)	Conf. + TACG	73.31 (+0.15)	69.01 (-5.62)	3.71 (+0.28)	
	d3LLM [†]	74.20	–	6.95	d3LLM [†]	77.20	–	4.80	
	KLASS	76.12	98.51	2.74	KLASS	79.68	155.96	1.64	
	KLASS + TACG	76.42 (+0.30)	106.66 (+8.15)	2.51 (-0.23)	KLASS + TACG	78.62 (-1.06)	91.94 (-64.02)	2.78 (+1.14)	

commitments is bounded, so the extra risk introduced by the temporal signal is explicitly limited per step.

A KL-style stability gate summarizes distribution-level change, whereas the TILG branch of TACG scores signed support for the proposed token. If $r_t(v) = \log p_t(v) - \log p_t^{\text{ref}}(v)$, a KL score aggregates r_t as $\mathbb{E}_{v \sim p_t}[r_t(v)]$, while TILG evaluates $r_t(\hat{x}_i)$ or a bounded monotone proxy for it. Thus, a distribution can still move globally while the current proposal is consistently reinforced; conversely, a distribution can look locally stable while no candidate gains positive temporal support.

4.6 From TACG to tokens per forward

Let $K_t^0 = |\mathcal{B}_t|$ and $\Delta_t = |\mathcal{E}_t|$. The number of tokens committed at step t is then $K_t^{\text{TACG}} = K_t^0 + \Delta_t$. For a target of N commits per block, the number of denoising steps satisfies $T = \min\{t : \sum_{\tau=1}^t K_\tau \geq N\}$, and tokens per forward can be approximated as $\text{TPF} \approx N/T$. When TACG increases K_t without introducing downstream errors that require additional recovery, T decreases and TPF increases. This benefit is not unconditional. If the base gate is already strong, the task is highly sensitive to early commitment, or the stability constraints are tight, TACG may primarily affect quality while producing smaller or task-dependent efficiency gains.

4.7 Complexity and overhead

TACG introduces no new parameters and no additional denoiser forward pass. Its per-step overhead consists of updating the EMA reference for TILG, computing temporal-support scores, enforcing the HG consistency checks, and selecting the top- K_{extra} extra candidates. The per-token tensor operations are $O(BLV)$, where B is the batch size, L is the block or sequence length, and V is the vocabulary size. The top- K_{extra} selection is applied over at most $|\mathcal{M}_t|$ candidates. A naive EMA reference also uses $O(BLV)$ memory. These costs are not negligible for large vocabularies, so step counts and tokens per forward should be interpreted as algorithmic efficiency indicators rather than complete system-throughput measurements.

5 Experiments

We evaluate TACG on three diffusion language models, LLaDA, Dream, and LLaDA2-Mini, across code and math benchmarks: HumanEval, MBPP, GSM8K, and MATH500. In each setting, TACG is plugged into the corresponding base decoder, and we report accuracy, denoising steps, and tokens per forward (TPF).

Table 2: LLaDA2-Mini results under two generation lengths. Green indicates improvements over baseline; red indicates declines. TACG denotes the full decoder, including the TILG temporal-support branch and the HG component.

Dataset	Method	gen_length = 1024		gen_length = 2048	
		Acc (%)	TPF	Acc (%)	TPF
HumanEval	Baseline	84.15	4.39	84.15	4.40
	TACG	85.98 (+1.83)	4.45 (+0.06)	85.37 (+1.22)	4.34 (-0.06)
MBPP	Baseline	80.09	2.73	80.09	2.74
	TACG	80.56 (+0.47)	2.82 (+0.09)	80.33 (+0.24)	2.76 (+0.02)
GSM8K	Baseline	87.64	2.04	88.32	2.03
	TACG	88.17 (+0.53)	2.04 (+0.00)	88.63 (+0.31)	2.03 (+0.00)
MATH500	Baseline	64.80	2.63	75.20	2.21
	TACG	67.40 (+2.60)	2.64 (+0.01)	76.40 (+1.20)	2.60 (+0.39)

Table 3: Unified component ablation across the three settings. ‘TACG (TILG+HG)’ denotes the full decoder that combines the TILG temporal-support branch with the HG persistence constraint.

Model / task	Decoder	Variant	Acc. ↑	Steps ↓	TPF ↑
LLaDA / HumanEval	Confidence	Baseline	37.20	52.28	6.4534
		HG only	38.41	60.89	5.3240
		TILG only	37.80	49.68	6.8359
		TACG (TILG+HG)	42.07	45.71	7.0136
LLaDA / HumanEval	KLASS	Baseline	39.63	92.13	3.3442
		HG only	39.63	114.61	2.6308
		TILG only	40.85	92.08	3.3696
		TACG (TILG+HG)	40.85	88.30	3.4914
Dream / MBPP	Confidence	Baseline	57.20	73.73	3.4721
		TILG only	56.42	68.36	3.7448
		HG only	57.59	70.97	3.6072
		TACG (TILG+HG)	57.59	70.93	3.6094

5.1 Main results on LLaDA

We first evaluate TACG on LLaDA under two representative decoding strategies: confidence-threshold decoding, which directly commits tokens whose confidence exceeds 0.9, and KLASS decoding[15], which uses KL-based stability signals for token selection. As shown in Table 1 (a), TACG under confidence decoding improves both quality and efficiency on all four benchmarks, with the clearest gain on HumanEval (37.20% → 42.07%, TPF 6.45 → 7.01). Under KLASS, accuracy still improves across all tasks, but the efficiency effect becomes task-dependent: TACG helps on HumanEval and MBPP, while the TPF changes on MATH500 and GSM8K are negligible or slightly negative. This is consistent with our intended interpretation. The TILG branch contributes readiness information that is complementary to KL-based stability, but TACG’s extra-promotion policy can interact conservatively with an already structured base gate. We also include d3LLM [26] as a recent competitive decoding baseline. Compared with d3LLM, TACG is often slower, but it is consistently more accurate across all four LLaDA tasks, highlighting a quality-oriented trade-off rather than a pure speed-first design.

5.2 Cross-model validation on Dream

We further evaluate TACG on Dream to test its transferability across dLLM backbones. As in the LLaDA experiments, TACG is added as a plug-and-play commit gate on top of confidence-threshold decoding and KLASS decoding. Table 1 (b) shows the same qualitative pattern on Dream. Under confidence decoding, TACG consistently reduces denoising steps and improves TPF while preserving or slightly improving accuracy, with the largest efficiency gain on MATH500. Under KLASS decoding, the effect is again task-dependent rather than uniformly accelerative: TACG speeds up MBPP and GSM8K substantially, and improves efficiency modestly on HumanEval and MATH500. GSM8K shows a small accuracy drop. This cross-model repetition is useful rather than redundant. It suggests that TACG behaves as a complementary readiness signal whose benefit depends on how much structure the base gate already imposes. Compared with d3LLM, TACG again trades some speed for stronger accuracy.

5.3 Stronger model: LLaDA2-Mini

We further evaluate TACG on LLaDA2-Mini, a stronger block diffusion model. Unlike full-sequence dLLMs, LLaDA2-Mini denoises one active block at a time, with previous blocks fixed as context. To adapt TACG, we maintain the historical logit reference within the active block, compute TILG temporal support along its block-local denoising trajectory, and apply the same HG logic within the active block, requiring no retraining or architectural modification. As shown in Table 2, TACG consistently improves accuracy under both generation lengths while broadly preserving TPF. The largest gains appear on MATH500, and the same direction of improvement holds across all four tasks at both 1024 and 2048 generation lengths. These results show that TACG naturally extends to block diffusion and remains effective on stronger dLLM backbones.

5.4 Ablation Study

Component analysis. We ablate TACG into its two main components, the TILG temporal-support branch and the HG persistence constraint, across the three settings in Table 3: LLaDA confidence decoding, LLaDA KLASS decoding, and Dream confidence decoding. Across all three settings,

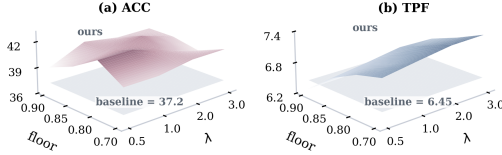


Figure 3: Sensitivity to support weight λ and confidence floor on HumanEval. TACG remains stable under appropriate parameter combinations.

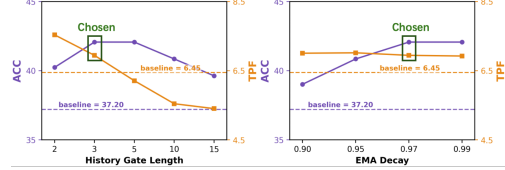


Figure 4: Sensitivity to TACG temporal-memory hyperparameters: History Gate (HG) length and EMA decay.

Table 4: LLaDA confidence decoding results on H200 with **gen=512**. Δ denotes TACG minus Baseline.

Dataset	Acc. $\uparrow$			Steps $\downarrow$			TPF $\uparrow$		
	Baseline	TACG	Δ	Baseline	TACG	Δ	Baseline	TACG	Δ
HumanEval	28.66	38.41	+9.75	103.82	94.15	-9.67	7.0230	7.1711	+0.1481
MBPP	41.63	45.14	+3.51	104.44	94.81	-9.63	5.6056	6.2096	+0.6040
GSM8K	80.21	82.79	+2.58	86.11	78.98	-7.13	6.4572	7.0476	+0.5904
MATH500	35.40	36.40	+1.00	122.98	119.25	-3.73	4.7644	4.9642	+0.1998

a clear division of labor emerges. TILG drives aggressive promotion and efficiency, whereas HG regularizes reliability through proposal persistence; the full TACG variant is where these two effects become balanced. Under LLaDA confidence decoding, TILG alone reduces steps and raises TPF, whereas HG alone improves accuracy but is visibly more conservative. The full variant achieves the best joint outcome (42.07% accuracy, 45.71 steps, 7.0136 TPF), which is the regime TACG is meant to occupy. Under LLaDA KLASS decoding, the same interaction persists in milder form. Because KLASS already imposes a distribution-level stability criterion, HG alone becomes overly conservative, TILG alone captures the accuracy gain, and the full variant further improves efficiency without sacrificing that accuracy. This supports the view that temporal support and KL-based stability are complementary rather than interchangeable.

On Dream confidence decoding, TILG alone gives the largest speedup but hurts accuracy, while HG recovers reliability. The full variant matches HG’s best accuracy while retaining most of the efficiency gain, reinforcing the same division of labor: TILG identifies candidates for earlier commitment, and HG filters unstable promotions.

Hyperparameter sensitivity. We analyze the joint effect of the support weight λ and the confidence floor τ_{floor} in Figure 3. The floor controls the candidate pool for commitment, while λ weights the readiness term $\lambda\sigma_i$ in our score. Across a broad range of settings, TACG remains stronger than the baseline, indicating that the method is not brittle. The main variation is in the quality–efficiency trade-off: moderate floor values favor accuracy, whereas larger λ values make promotion more aggressive and therefore raise TPF. We further analyze the HG window length and the EMA decay in Figure 4. Moderate HG windows provide the best ACC–TPF trade-off, whereas overly long windows become conservative and reduce TPF. Larger β improves ACC with relatively stable TPF, indicating that the TILG branch benefits from a smoother historical reference.

5.5 Generalization to longer generation length

We further evaluate LLaDA confidence decoding with **generation length=512** on H200 to test whether the gains of TACG persist under a longer generation budget. As shown in Table 4, the improvement remains consistent across all four benchmarks. The largest gain appears on HumanEval, where accuracy rises from 28.66% to 38.41% while steps fall from 103.82 to 94.15; MBPP and GSM8K also show clear TPF gains. Overall, TACG is not limited to short-generation settings and continues to improve quality and decoding parallelism when the generation budget is extended.

6 Conclusion

We introduced TACG, a training-free gate-level decoder that estimates commitment readiness from the DLLM denoising trajectory by combining TILG temporal support with an HG persistence constraint, while keeping token identity anchored to the base posterior. Empirically, TACG improves commitment timing without changing token identity, is most effective under confidence-based gates, and remains complementary to distribution-level stability rules such as KLASS.

References

- [1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- [2] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces, 2023. URL <https://arxiv.org/abs/2107.03006>.
- [3] Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, Zenan Huang, Zhenzhong Lan, Chengxi Li, Chongxuan Li, Jianguo Li, Zehuan Li, Huabin Liu, Lin Liu, Guoshan Lu, Xiaocheng Lu, Yuxin Ma, Jianfeng Tan, Lanning Wei, Ji-Rong Wen, Yipeng Xing, Xiaolu Zhang, Junbo Zhao, Da Zheng, Jun Zhou, Junlin Zhou, Zhanchao Zhou, Liwang Zhu, and Yihong Zhuang. Llada2.0: Scaling up diffusion language models to 100b, 2025. URL <https://arxiv.org/abs/2512.15745>.
- [4] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. Maskgit: Masked generative image transformer, 2022. URL <https://arxiv.org/abs/2202.04200>.
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgens Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [6] Shuang Cheng, Yuhua Jiang, Zineng Zhou, Dawei Liu, Wang Tao, Linfeng Zhang, Biqing Qi, and Bowen Zhou. Sdar-vl: Stable and efficient block-wise diffusion for vision-language understanding, 2025. URL <https://arxiv.org/abs/2512.14068>.
- [7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [8] Chenghao Fan, Wen Heng, Bo Li, Sichen Liu, Yuxuan Song, Jing Su, Xiaoye Qu, Kai Shen, and Wei Wei. Stable-diffcoder: Pushing the frontier of code diffusion large language model, 2026. URL <https://arxiv.org/abs/2601.15892>.
- [9] Shansan Gong, Ruixiang Zhang, Huangjie Zheng, Jiatao Gu, Navdeep Jaitly, Lingpeng Kong, and Yizhe Zhang. Diffucoder: Understanding and improving masked diffusion models for code generation, 2025. URL <https://arxiv.org/abs/2506.20639>.
- [10] Guangxin He, Shen Nie, Fengqi Zhu, Yuankang Zhao, Tianyi Bai, Ran Yan, Jie Fu, Chongxuan Li, and Binhang Yuan. Ultrallada: Scaling the context length to 128k for diffusion large language models, 2025. URL <https://arxiv.org/abs/2510.10481>.
- [11] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.
- [12] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance, 2022. URL <https://arxiv.org/abs/2207.12598>.
- [13] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. URL <https://arxiv.org/abs/2006.11239>.

- [14] Daniel Israel, Guy Van den Broeck, and Aditya Grover. Accelerating diffusion llms via adaptive parallel decoding, 2025. URL <https://arxiv.org/abs/2506.00413>.
- [15] Seo Hyun Kim, Sunwoo Hong, Hojung Jung, Youngrok Park, and Se-Young Yun. Klass: Kl-guided fast inference in masked diffusion models, 2026. URL <https://arxiv.org/abs/2511.05664>.
- [16] Fanheng Kong, Jingyuan Zhang, Yahui Liu, Zirui Wu, Yu Tian, Victoria W., and Guorui Zhou. Accelerating diffusion llm inference via local determinism propagation, 2025. URL <https://arxiv.org/abs/2510.07081>.
- [17] Sanghyun Lee, Seungryong Kim, Jongho Park, and Dongmin Park. Lookahead unmasking elicits accurate decoding in diffusion language models, 2025. URL <https://arxiv.org/abs/2511.05563>.
- [18] Pengxiang Li, Yefan Zhou, Dilxat Muhtar, Lu Yin, Shilin Yan, Li Shen, Soroush Vosoughi, and Shiwei Liu. Diffusion language models know the answer before decoding, 2026. URL <https://arxiv.org/abs/2508.19982>.
- [19] Xiaoran Liu, Yuerong Song, Zhigeng Liu, Zengfeng Huang, Qipeng Guo, Ziwei He, and Xipeng Qiu. Longllada: Unlocking long context capabilities in diffusion llms, 2025. URL <https://arxiv.org/abs/2506.14429>.
- [20] Yang Liu, Pengxiang Ding, Tengyue Jiang, Xudong Wang, Wenxuan Song, Minghui Lin, Han Zhao, Hongyin Zhang, Zifeng Zhuang, Wei Zhao, Siteng Huang, Jinkui Shi, and Donglin Wang. Mmada-vla: Large diffusion vision-language-action model with unified multi-modal instruction and generation, 2026. URL <https://arxiv.org/abs/2603.25406>.
- [21] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution, 2024. URL <https://arxiv.org/abs/2310.16834>.
- [22] Zanlin Ni, Shenzhi Wang, Yang Yue, Tianyu Yu, Weilin Zhao, Yeguo Hua, Tianyi Chen, Jun Song, Cheng Yu, Bo Zheng, and Gao Huang. The flexibility trap: Rethinking the value of arbitrary order in diffusion language models, 2026. URL <https://arxiv.org/abs/2601.15165>.
- [23] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models, 2025. URL <https://arxiv.org/abs/2502.09992>.
- [24] Leyi Pan, Shuchang Tao, Yunpeng Zhai, Zheyu Fu, Liancheng Fang, Minghua He, Lingzhe Zhang, Zhaoyang Liu, Bolin Ding, Aiwei Liu, and Lijie Wen. d-treerpo: Towards more reliable policy optimization for diffusion language models, 2026. URL <https://arxiv.org/abs/2512.09675>.
- [25] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. Sdxl: Improving latent diffusion models for high-resolution image synthesis, 2023. URL <https://arxiv.org/abs/2307.01952>.
- [26] Yu-Yang Qian, Junda Su, Lanxiang Hu, Peiyuan Zhang, Zhijie Deng, Peng Zhao, and Hao Zhang. d3llm: Ultra-fast diffusion llm using pseudo-trajectory distillation, 2026. URL <https://arxiv.org/abs/2601.07568>.
- [27] Kevin Rojas, Jiahe Lin, Kashif Rasul, Anderson Schneider, Yuriy Nevmyvaka, Molei Tao, and Wei Deng. Improving reasoning for diffusion language models via group diffusion policy optimization, 2026. URL <https://arxiv.org/abs/2510.08554>.
- [28] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models, 2022. URL <https://arxiv.org/abs/2112.10752>.
- [29] Nataniel Ruiz, Yuanzhen Li, Varun Jampani, Yael Pritch, Michael Rubinstein, and Kfir Aberman. Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation, 2023. URL <https://arxiv.org/abs/2208.12242>.

- [30] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models, 2024. URL <https://arxiv.org/abs/2406.07524>.
- [31] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data, 2025. URL <https://arxiv.org/abs/2406.04329>.
- [32] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models, 2022. URL <https://arxiv.org/abs/2010.02502>.
- [33] Xiaohang Tang, Rares Dolga, Sangwoong Yoon, and Ilija Bogunovic. wdl: Weighted policy optimization for reasoning in diffusion language models, 2026. URL <https://arxiv.org/abs/2507.08838>.
- [34] Yuqing Wen, Hebei Li, Kefan Gu, Yucheng Zhao, Tiancai Wang, and Xiaoyan Sun. Llada-vla: Vision language diffusion action models, 2025. URL <https://arxiv.org/abs/2509.06932>.
- [35] Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding, 2025. URL <https://arxiv.org/abs/2505.22618>.
- [36] Zhongyu Xiao, Zhiwei Hao, Jianyuan Guo, Yong Luo, Jia Liu, Jie Xu, and Han Hu. Streaming-dllm: Accelerating diffusion llms via suffix pruning and dynamic decoding, 2026. URL <https://arxiv.org/abs/2601.17917>.
- [37] Shaoan Xie, Lingjing Kong, Xiangchen Song, Xinshuai Dong, Guangyi Chen, Eric P. Xing, and Kun Zhang. Advancing reasoning in diffusion language models with denoising process rewards, 2026. URL <https://arxiv.org/abs/2510.01544>.
- [38] Zhihui Xie, Jiacheng Ye, Lin Zheng, Jiahui Gao, Jingwei Dong, Zirui Wu, Xueliang Zhao, Shansan Gong, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream-coder 7b: An open diffusion language model for code, 2025. URL <https://arxiv.org/abs/2509.01142>.
- [39] Chenkai Xu, Yijie Jin, Jiajun Li, Yi Tu, Guoping Long, Dandan Tu, Mingcong Song, Hongjie Si, Tianqi Hou, Junchi Yan, and Zhijie Deng. Lopa: Scaling dllm inference via lookahead parallel decoding, 2025. URL <https://arxiv.org/abs/2512.16229>.
- [40] Ling Yang, Ye Tian, Bowen Li, Xincheng Zhang, Ke Shen, Yunhai Tong, and Mengdi Wang. Mmada: Multimodal large diffusion language models, 2025. URL <https://arxiv.org/abs/2505.15809>.
- [41] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models, 2025. URL <https://arxiv.org/abs/2508.15487>.
- [42] Jiacheng Ye, Shansan Gong, Jiahui Gao, Junming Fan, Shuang Wu, Wei Bi, Haoli Bai, Lifeng Shang, and Lingpeng Kong. Dream-vl & dream-vla: Open vision-language and vision-language-action models with diffusion language model backbone, 2026. URL <https://arxiv.org/abs/2512.22615>.
- [43] Zebin You, Shen Nie, Xiaolu Zhang, Jun Hu, Jun Zhou, Zhiwu Lu, Ji-Rong Wen, and Chongxuan Li. Llada-v: Large language diffusion models with visual instruction tuning, 2025. URL <https://arxiv.org/abs/2505.16933>.
- [44] Runpeng Yu, Xinyin Ma, and Xinchao Wang. Dimple: Discrete diffusion multimodal large language model with parallel decoding, 2025. URL <https://arxiv.org/abs/2505.16990>.
- [45] Lunbin Zeng, Jingfeng Yao, Bencheng Liao, Hongyuan Tao, Wenyu Liu, and Xinggang Wang. Diffusionvl: Translating any autoregressive models into diffusion vision language models, 2026. URL <https://arxiv.org/abs/2512.15713>.

- [46] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models, 2023. URL <https://arxiv.org/abs/2302.05543>.
- [47] Jiahao Zhao, Shaoxuan Xu, Zhongxiang Sun, Fengqi Zhu, Jingyang Ou, Yuling Shi, Chongxuan Li, Xiao Zhang, and Jun Xu. Dllm-searcher: Adapting diffusion large language model for search agents, 2026. URL <https://arxiv.org/abs/2602.07035>.
- [48] Siyan Zhao, Devaansh Gupta, Qinqing Zheng, and Aditya Grover. d1: Scaling reasoning in diffusion large language models via reinforcement learning, 2025. URL <https://arxiv.org/abs/2504.12216>.
- [49] Huiling Zhen, Weizhe Lin, Renxi Liu, Kai Han, Yiming Li, Yuchuan Tian, Hanting Chen, Xiaoguang Li, Xiaosong Li, Chen Chen, Xianzhi Yu, Mingxuan Yuan, Youliang Yan, Peifeng Qin, Jun Wang, Yu Wang, Dacheng Tao, and Yunhe Wang. Dllm agent: See farther, run faster, 2026. URL <https://arxiv.org/abs/2602.07451>.
- [50] Kaiwen Zheng, Yongxin Chen, Hanzi Mao, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Masked diffusion models are secretly time-agnostic masked models and exploit inaccurate categorical sampling, 2025. URL <https://arxiv.org/abs/2409.02908>.
- [51] Liang Zheng, Bowen Shi, Yitao Hu, Jiawei Zhang, Ruofan Li, Sheng Chen, Wenxin Li, and Keqiu Li. Mosaic: Unlocking long-context inference for diffusion llms via global memory planning and dynamic peak taming, 2026. URL <https://arxiv.org/abs/2601.06562>.
- [52] Yuyan Zhou, Kai Syun Hou, Weiyu Chen, and James Kwok. Attention-based sampler for diffusion language models, 2026. URL <https://arxiv.org/abs/2604.08564>.
- [53] Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Llada 1.5: Variance-reduced preference optimization for large language diffusion models, 2025. URL <https://arxiv.org/abs/2505.19223>.
- [54] Fengqi Zhu, Zebin You, Yipeng Xing, Zenan Huang, Lin Liu, Yihong Zhuang, Guoshan Lu, Kangyu Wang, Xudong Wang, Lanning Wei, Hongrui Guo, Jiaqi Hu, Wentao Ye, Tieyuan Chen, Chenchen Li, Chengfu Tang, Haibo Feng, Jun Hu, Jun Zhou, Xiaolu Zhang, Zhenzhong Lan, Junbo Zhao, Da Zheng, Chongxuan Li, Jianguo Li, and Ji-Rong Wen. Llada-moe: A sparse moe diffusion language model, 2025. URL <https://arxiv.org/abs/2509.24389>.

A Additional experimental results

A.1 Decoding dynamics

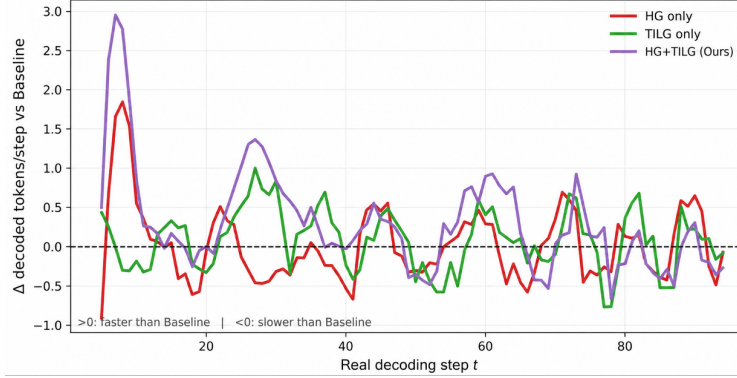


Figure 5: Per-step decoding dynamics on HumanEval. Positive values indicate more decoded tokens per step than the baseline.

Figure 5 reports the per-step difference in committed tokens relative to the baseline, so positive values indicate faster commitment. TILG alone gives limited early gain, whereas full TACG shows a pronounced early peak. This reinforces the ablation story: HG is not merely a conservative filter, but the mechanism that turns temporal support into coherent early commitment. The later fluctuations around zero suggest that TACG selectively advances stable regions rather than uniformly increasing commitment.

Table 5: Latency-only comparison (**Steps** as latency proxy; lower is better). Confidence and KLASS are treated as two separate baselines.

Dataset	Baseline Type	LLaDA		Dream	
		Base Steps	TACG Steps ($\Delta\%$)	Base Steps	TACG Steps ($\Delta\%$)
HumanEval	Confidence	52.28	45.71 (-12.57%)	51.19	49.45 (-3.40%)
	KLASS	92.13	88.30 (-4.16%)	75.09	72.05 (-4.05%)
MBPP	Confidence	83.34	74.76 (-10.30%)	73.73	70.93 (-3.80%)
	KLASS	121.08	119.70 (-1.14%)	111.25	82.17 (-26.14%)
MATH500	Confidence	96.48	86.59 (-10.25%)	94.60	80.33 (-15.08%)
	KLASS	127.70	133.33 ($+4.41\%$)	148.85	145.79 (-2.06%)
GSM8K	Confidence	74.26	72.29 (-2.65%)	74.63	69.01 (-7.53%)
	KLASS	98.51	106.66 ($+8.27\%$)	155.96	91.94 (-41.05%)

B Additional theoretical details

This appendix records the formal arguments that support the interpretation of TACG in Section 4, especially its TILG temporal-support branch. These statements are intended as principled interpretations of the decoder, not as claims that TACG recovers the true posterior or solves an optimal stopping problem.

B.1 DLLM decoding is a joint stopping-and-labeling problem

At denoising step t , a decoder observes the masked set $\mathcal{M}_t$ and must decide, for each $i \in \mathcal{M}_t$, both whether the position should leave the masked state and which token should be written if it does. Formally, the action is

$$\mathcal{A}_t = \{(a_{t,i}, y_{t,i})\}_{i \in \mathcal{M}_t}, \quad a_{t,i} \in \{0, 1\}, \quad y_{t,i} \in \mathcal{V}. \quad (20)$$

If $a_{t,i}$ is omitted, the decoder has no rule for when to reveal a position. If $y_{t,i}$ is omitted, the decoder has no label to write after deciding to reveal. DLLM decoding therefore couples a stopping decision with a labeling decision. This decomposition matters because many decoding rules use the same snapshot distribution to answer both questions, even though the stopping decision is the part most exposed to incomplete context.

B.2 Logits are natural coordinates for categorical belief

For a categorical predictive distribution parameterized by logits z ,

$$p(v) = \frac{\exp z(v)}{\sum_u \exp z(u)}. \quad (21)$$

For any two tokens a and b ,

$$\log \frac{p(a)}{p(b)} = z(a) - z(b). \quad (22)$$

Thus logit differences are exactly log-odds differences. Tracking a trajectory in logit space amounts to tracking how the model moves in the natural-parameter coordinates of the categorical family. This motivates constructing the TILG self-reference before the softmax, rather than storing only hard token histories or scalar confidences.

B.3 Temporal support is a historical log-ratio signal

For derivation, write the auxiliary readout scores as

$$u_t = (1 + w)z_t - w\bar{z}_{t-1}. \quad (23)$$

Let $p_t = \text{softmax}(z_t)$, $p_t^{\text{ref}} = \text{softmax}(\bar{z}_{t-1})$, and $q_t = \text{softmax}(u_t)$. Since logits are identifiable only up to an additive constant, we can write

$$\exp u_t(v) = \exp((1 + w)z_t(v) - w\bar{z}_{t-1}(v)) \quad (24)$$

$$\propto \frac{p_t(v)^{1+w}}{p_t^{\text{ref}}(v)^w}. \quad (25)$$

After normalization,

$$q_t(v) \propto p_t(v) \left(\frac{p_t(v)}{p_t^{\text{ref}}(v)} \right)^w. \quad (26)$$

The auxiliary readout therefore exposes a likelihood ratio between the current belief and the historical reference. Tokens whose support increases relative to the reference receive higher temporal support, while tokens with high current probability but weakening historical support receive less promotion by the gate.

B.4 First-order belief-innovation interpretation

Let $d_t = z_t - \bar{z}_{t-1}$ and $q_w = \text{softmax}(z_t + wd_t)$. At $w = 0$, $q_w = p_t$. The derivative of softmax gives

$$\left. \frac{\partial q_w(v)}{\partial w} \right|_{w=0} = p_t(v) (d_t(v) - \mathbb{E}_{u \sim p_t}[d_t(u)]). \quad (27)$$

Thus a small increase in w raises the auxiliary readout value of tokens whose belief innovation is above the current average and lowers the readout value of tokens whose innovation is below that average. This supports the use of q_t as a directional temporal signal rather than a simple confidence-sharpening operation.

B.5 Gauge invariance

Categorical logits are not uniquely identified: adding a token-independent constant does not change the softmax. Suppose $z'_t(v) = z_t(v) + a$ and $\bar{z}'_{t-1}(v) = \bar{z}_{t-1}(v) + b$ for constants a and b that do not depend on token v . Then

$$u'_t(v) = (1 + w)z'_t(v) - w\bar{z}'_{t-1}(v) = u_t(v) + (1 + w)a - wb. \quad (28)$$

The added term is identical for every token, so q_t is unchanged after normalization. The temporal support statistic is therefore invariant to the additive gauge of logits at each position.

B.6 Difference from KL-style stability signals

Define the token-wise log-ratio

$$r_t(v) = \log p_t(v) - \log p_t^{\text{ref}}(v). \quad (29)$$

A KL-style stability score aggregates this signal over the whole distribution,

$$D_{\text{KL}}(p_t \| p_t^{\text{ref}}) = \mathbb{E}_{v \sim p_t} [r_t(v)]. \quad (30)$$

This quantity measures distribution-level change. The TILG branch of TACG, by contrast, uses the candidate-conditioned signal

$$r_t(\hat{x}_i) = \log p_t(i, \hat{x}_i) - \log p_t^{\text{ref}}(i, \hat{x}_i), \quad (31)$$

or a bounded monotone proxy for it. The two signals answer different questions. KL-style stability asks whether the whole distribution is still changing. TILG support asks whether the proposed token is gaining signed support relative to history. A distribution may change substantially while the current proposal is consistently reinforced, and a distribution may remain nearly unchanged while no candidate becomes more commit-ready. The signals are therefore complementary.

B.7 Capped extra promotion as bounded acceleration

The extra promotion set satisfies $|\mathcal{E}_t| \leq K_{\text{extra}}$. If the conditional error probability of promoted candidates at step t is bounded by ϵ_t , then the expected number of additional errors introduced by extra promotion at that step is bounded by

$$\mathbb{E}[\text{extra errors}_t] \leq K_{\text{extra}} \epsilon_t. \quad (32)$$

This bound is intentionally loose and should not be interpreted as a formal safety guarantee. Its purpose is to clarify the role of K_{extra} : temporal support can move the reveal boundary, but each step has a fixed budget for additional risk.

C Implementation sketch

Figure 6 gives a compact implementation sketch. It follows the default experimental setting: the base posterior proposes token identity, the TILG EMA reference supplies temporal support, the HG stabilizes base acceptance, and extra promotion is lightly constrained by proposal persistence or a confidence escape.

```
class TACGCommitState:
    def __init__(self):
        self.z_ref = None # EMA / previous-logit self-reference
        self.prev_proposal = None # previous base-posterior token proposal
        self.proposal_streak = None # consecutive identical proposals

    def reference(self, logits):
        if self.z_ref is None:
            self.z_ref = detach(logits) # neutral first TACG step
        return self.z_ref

    def update_streak(self, x_hat, mask):
        if self.prev_proposal is None:
            self.proposal_streak = ones_like(x_hat)
        else:
            same_proposal = (x_hat == self.prev_proposal)
            self.proposal_streak = where(mask & same_proposal,
                                         self.proposal_streak + 1, 1)
            self.prev_proposal = detach(x_hat)

    def update_reference(self, logits, beta):
        self.z_ref = beta * self.z_ref + (1.0 - beta) * detach(logits)

def tacg_commit_step(logits, mask, base_gate, state, cfg):
    # Base posterior proposes token identity.
    p = softmax(logits)
    x_hat = argmax(p, dim=-1)
    confidence = gather(p, x_hat)
    state.update_streak(x_hat, mask)

    # TILG computes temporal support for the base proposal.
    ref_logits = state.reference(logits)
    readout_scores = logits + cfg.w * (logits - ref_logits)
    support_readout = softmax(readout_scores)
    p_ref = softmax(ref_logits)
    support = clamp(gather(support_readout, x_hat)
                   - gather(p_ref, x_hat), min=0.0)
    readiness = confidence + cfg.support_lambda * support

    # The underlying decoder supplies the base commit candidates
    # (e.g., confidence threshold or KLASS stability + confidence).
    base_ready = base_gate(p, logits, x_hat, confidence, mask)
    base_consistency_ok = ((state.proposal_streak >= cfg.m_base)
                          | (confidence >= cfg.tau_escape))
    base_ready = base_ready & base_consistency_ok & mask

    # TACG only moves the commitment boundary under a capped budget.
    extra_consistency_ok = ((state.proposal_streak >= cfg.m_extra)
                          | (confidence >= cfg.tau_escape))
    extra_candidates = (mask & ~base_ready
                       & (confidence >= cfg.tau_floor)
                       & extra_consistency_ok)
    extra_ready = masked_topk(readiness, extra_candidates, k=cfg.K_extra)

    commit_mask = base_ready | extra_ready
    write_tokens = where(commit_mask, x_hat, MASK)

    state.update_reference(logits, beta=cfg.beta)
    return commit_mask, write_tokens, {
        "confidence": confidence,
        "temporal_support": support,
        "readiness": readiness,
    }
```

Figure 6: Python-style pseudocode for TACG. The TILG branch computes temporal support, the HG-style consistency checks stabilize base acceptance and extra promotion, and the token identity always comes from the base posterior.

A log-ratio support variant replaces the probability-gain support computation with

$$\sigma_i = \text{clip} \left(\log(p_t(i, \hat{x}_i) + \epsilon) - \log(p_t^{\text{ref}}(i, \hat{x}_i) + \epsilon), 0, \gamma \right). \quad (33)$$

This variant is closer to the derivation in Appendix B, while the probability-gain version is bounded in $[0, 1]$ and is used as the default implementation in the current experiments.

D Limitations

TACG is designed as a lightweight decoding-layer commit policy for commitment readiness. In the present work, its TILG branch uses q_t as an operational temporal-support statistic while keeping token identity anchored in the base posterior. This separation is intentional: it preserves the calibration and vocabulary preference of the pretrained decoder, and uses temporal evidence only to adjust the commitment boundary.

The current formulation scores each masked position independently, which makes the method simple to plug into existing DLLM decoders. This modularity also suggests future extensions. TACG can be combined with dependency-aware schedulers, learned unmasking policies, or group-level commit rules so that mutually dependent tokens are promoted in a more coordinated way. On the systems side, TACG requires no additional denoiser forward pass, and future implementations can further reduce tensor overhead through compact reference states, fused softmax/top- K kernels, and detailed wall-clock and memory profiling.

E Broader Impact

TACG may improve the efficiency and reliability of diffusion language model decoding without retraining, which can reduce inference cost for benign text-generation applications. As with other improvements to language generation, the same capability could also be used to generate lower-cost misleading or harmful content; deployment should therefore follow the safeguards of the underlying models and applications.